
Session 11 (AIML) - Assignment Questions

Name: **Vaishnavi Praveen Rathi**

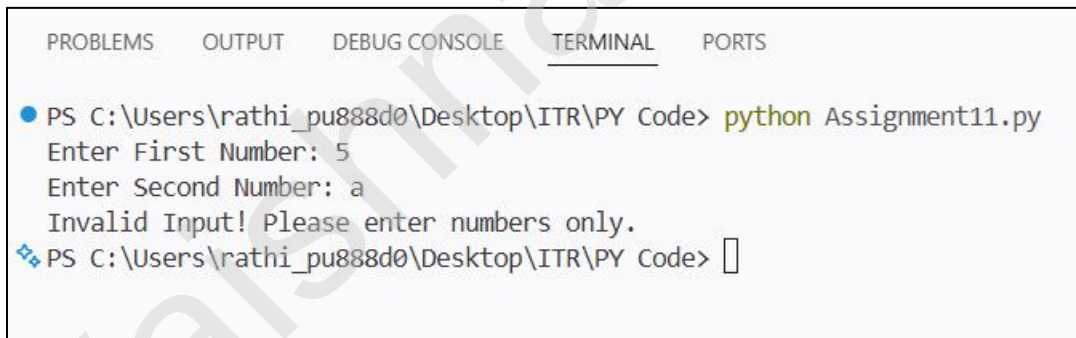
College: **Zeal Polytechnic, Narhe, Pune**

Branch: **Artificial Intelligence and Machine Learning (AIML)**

Domain: **AIML (Artificial Intelligence and Machine Learning)**

Q1. Basic try-except

Output Screenshot:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> python Assignment11.py
Enter First Number: 5
Enter Second Number: a
Invalid Input! Please enter numbers only.
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> 
```

Program Code:

```
""" ASSIGNMENT 11 """
```

```
'''
```

Q1. (Basic try-except)

Write a program that takes two numbers as input from the user and prints their division.

Use try-except to handle the case when the user enters non-numeric input (ValueError).

Print a friendly message if an error occurs.

```
'''
```

try:

```
# Take input from the user
num1 = float(input("Enter First Number: "))
num2 = float(input("Enter Second Number: "))
```

```
# Perform division
result = num1 / num2
```

```
# Display result
print("Division =", result)
```

except ValueError:

```
# Handle non-numeric input
print("Invalid Input! Please enter numbers only.")
```

"""Explanation in Comments"""

```
# The try block contains code that may cause an error.
# float() converts user input into numbers.
# If the user enters text instead of a number,
# ValueError occurs.
# The except block catches the error and displays
# a friendly message instead of crashing the program.
```

Q2. Handling ZeroDivisionError

Output Screenshot:



The screenshot shows a terminal window with tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The TERMINAL tab is active. The prompt is PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code>. The user enters 'python Assignment11.py'. The program prompts 'Enter First Number: 10' and 'Enter Second Number: 0'. It then prints 'Error: Division by zero is not allowed.' and returns to the prompt.

```
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> python Assignment11.py
Enter First Number: 10
Enter Second Number: 0
Error: Division by zero is not allowed.
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> 
```

Program Code:

```
""" ASSIGNMENT 11 """
```

```
'''
```

Q2. (Handling ZeroDivisionError)

Write a program that divides two numbers entered by the user.

Use try-except to specifically catch ZeroDivisionError and print:
"Error: Division by zero is not allowed."

Also handle ValueError for invalid input.

```
'''
```

```
try:
```

```
    # Take input from the user
```

```
    num1 = float(input("Enter First Number: "))
```

```
    num2 = float(input("Enter Second Number: "))
```

```
    # Perform division
```

```
    result = num1 / num2
```

```
    # Display result
```

```
    print("Division =", result)
```

```
except ZeroDivisionError:
```

```
    # Handle division by zero
```

```
print("Error: Division by zero is not allowed.")

except ValueError:

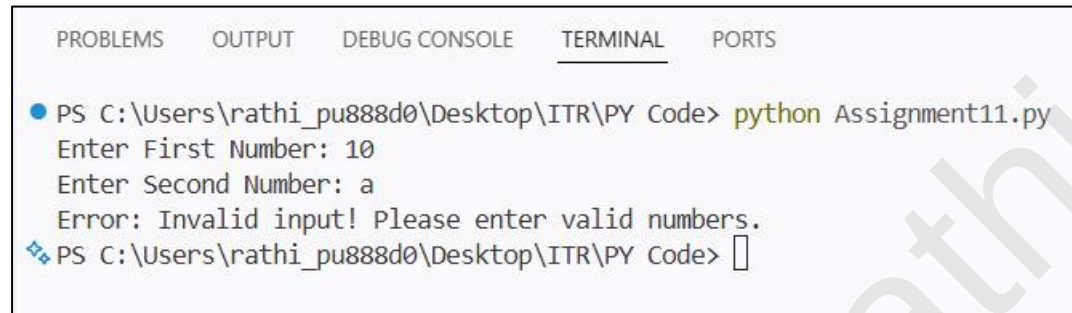
    # Handle invalid input
    print("Invalid Input! Please enter numbers only.")
```

"Explanation in Comments"

```
# The try block contains code that may generate errors.
# If the second number is 0, ZeroDivisionError occurs.
# If the user enters non-numeric input, ValueError occurs.
# Separate except blocks handle each error individually.
# This prevents the program from crashing and shows
# user-friendly error messages.
```

Q3. Multiple except blocks

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> python Assignment11.py
Enter First Number: 10
Enter Second Number: a
Error: Invalid input! Please enter valid numbers.
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> 
```

Program Code:

```
""" ASSIGNMENT 11 """
```

```
'''
```

Q3. (Multiple except Blocks)

Create a program that performs division and also tries to convert a string to integer.

Handle both ValueError and ZeroDivisionError with separate except blocks and appropriate messages.

```
'''
```

```
try:
```

```
    # Take two numbers from the user
```

```
    num1 = float(input("Enter First Number: "))
```

```
    num2 = float(input("Enter Second Number: "))
```

```
    # Perform division
```

```
    result = num1 / num2
```

```
    print("Division =", result)
```

```
    # Convert string input to integer
```

```
    value = int(input("Enter an Integer Value: "))
```

```
    print("Integer Value =", value)
```

```
except ZeroDivisionError:
```

```
# Handle division by zero
print("Error: Division by zero is not allowed.")

except ValueError:

    # Handle invalid numeric input
    print("Error: Invalid input! Please enter valid numbers.")
```

"Explanation in Comments"

```
# The try block contains code that may generate errors.
# Division is performed using two user-entered numbers.
# int() is used to convert input into an integer.
# ZeroDivisionError occurs when dividing by zero.
# ValueError occurs when non-numeric data is entered.
# Separate except blocks handle each error individually.
# This makes the program more user-friendly and prevents crashes.
```

Q4. Using else

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> python Assignment11.py
Enter First Number: 10
Enter Second Number: 2
Division = 5.0
Division performed successfully!
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> 
```

Program Code:

```
""" ASSIGNMENT 11 """
```

```
'''
```

Q4. (Using else)

Write a program that takes two numbers and divides them.

Use try, except, and else.

The else block should print
"Division performed successfully!"
only when no exception occurs.

```
'''
```

```
try:
```

```
    # Take input from the user
```

```
    num1 = float(input("Enter First Number: "))
```

```
    num2 = float(input("Enter Second Number: "))
```

```
    # Perform division
```

```
    result = num1 / num2
```

```
except ZeroDivisionError:
```

```
    # Handle division by zero
```

```
    print("Error: Division by zero is not allowed.")
```

except ValueError:

```
# Handle invalid input
print("Error: Please enter valid numbers.")
```

else:

```
# Executes only if no exception occurs
print("Division =", result)
print("Division performed successfully!")
```

"Explanation in Comments"

```
# The try block contains code that may generate exceptions.
# ZeroDivisionError occurs when dividing by zero.
# ValueError occurs when non-numeric input is entered.
# The else block runs only when no exception occurs.
# It displays the result and a success message.
# This improves program readability and error handling.
```

Q5 Using finally

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> python Assignment11.py
Enter First Number: 10
Enter Second Number: 5
Division = 2.0
Program execution completed.
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> 
```

Program Code:

```
""" ASSIGNMENT 11 """
```

```
'''
```

Q5. (Using finally)

Modify the division program to include a finally block that always prints:
"Program execution completed."

Demonstrate that finally runs whether an exception occurs or not.

Add comments explaining the purpose of finally.

```
'''
```

```
try:
```

```
    # Take input from the user
```

```
    num1 = float(input("Enter First Number: "))
```

```
    num2 = float(input("Enter Second Number: "))
```

```
    # Perform division
```

```
    result = num1 / num2
```

```
    print("Division =", result)
```

```
except ZeroDivisionError:
```

```
    # Handle division by zero
```

```
print("Error: Division by zero is not allowed.")

except ValueError:

    # Handle invalid input
    print("Error: Please enter valid numbers.")

finally:

    # This block always executes
    # whether an exception occurs or not
    print("Program execution completed.")
```

"Explanation in Comments"

```
# The try block contains code that may generate exceptions.
# ZeroDivisionError occurs when dividing by zero.
# ValueError occurs when non-numeric input is entered.
# The finally block always executes.
# It runs whether an exception occurs or not.
# finally is commonly used for cleanup tasks such as
# closing files, database connections, or releasing resources.
```

Q6. Combined try-except-else-finally

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> python Assignment11.py
Enter First Number: 10
Enter Second Number: 8
Division = 1.25
Division performed successfully!
Thank you for using the program.
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> 
```

Program Code:

```
""" ASSIGNMENT 11 """
```

```
'''
```

Q6. (Combined try-except-else-finally)

Write a complete division program using all four blocks (try, except, else, finally):

Handle ValueError and ZeroDivisionError.

Print success message in else.

Always thank the user in finally.

```
'''
```

```
try:
```

```
    # Take input from the user
```

```
    num1 = float(input("Enter First Number: "))
```

```
    num2 = float(input("Enter Second Number: "))
```

```
    # Perform division
```

```
    result = num1 / num2
```

```
except ZeroDivisionError:
```

```
    # Handle division by zero
```

```
    print("Error: Division by zero is not allowed.")
```

```
except ValueError:
```

```
# Handle invalid input
print("Error: Please enter valid numbers.")

else:

    # Executes only when no exception occurs
    print("Division =", result)
    print("Division performed successfully!")

finally:

    # Executes in all cases
    print("Thank you for using the program.")
```

"Explanation in Comments"

```
# try block contains code that may generate exceptions.
# except blocks handle specific errors.
# else block runs only if no exception occurs.
# finally block always runs, regardless of success or error.
# finally is useful for cleanup tasks and closing resources.
```

Q7. Practical - Age Input

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu88d0\Desktop\ITR\PY Code> python Assignment11.py
Enter Your Age: -17
Error: Age cannot be negative.
PS C:\Users\rathi_pu88d0\Desktop\ITR\PY Code> 
```

Program Code:

```
""" ASSIGNMENT 11 """

"""
Q7. (Practical - Age Input)

Write a program that asks the user for their age.

Convert the input to integer.

If the age is negative, raise a ValueError with message
"Age cannot be negative."

Handle the exception gracefully and print a suitable message.
"""

try:
    # Take age as input
    age = int(input("Enter Your Age: "))

    # Check if age is negative
    if age < 0:
        raise ValueError("Age cannot be negative.")

    print("Your Age is:", age)

except ValueError as error:

    # Handle invalid or negative age
    print("Error:", error)
```

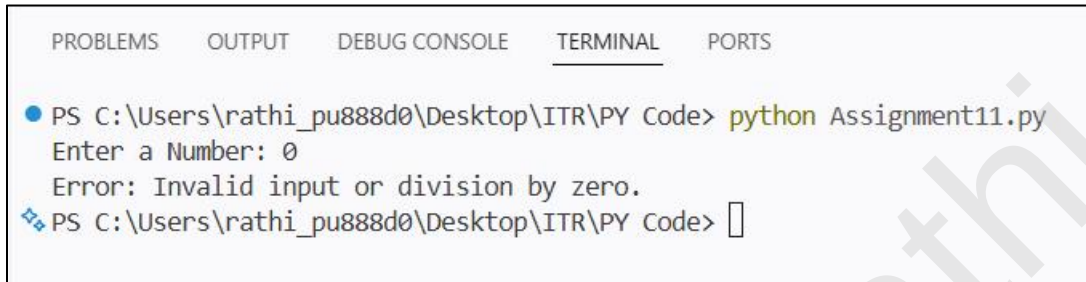
"Explanation in Comments"

```
# The program accepts age from the user.  
# int() converts the input into an integer.  
# If age is less than 0, ValueError is raised manually.  
# raise keyword is used to generate a custom exception.  
# The except block handles the error and displays a message.  
# This prevents the program from crashing.
```

Vaishnavi Rathni

Q8. Multiple Exceptions in one block

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> python Assignment11.py
Enter a Number: 0
Error: Invalid input or division by zero.
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> 
```

Program Code:

```
""" ASSIGNMENT 11 """
```

```
'''
```

Q8. (Multiple Exceptions in One Block)

Create a program that:

Takes a number as input.

Divides 100 by that number.

Handle both ValueError and ZeroDivisionError
in a single except block using a tuple.

Print a combined error message.

```
'''
```

```
try:
```

```
    # Take input from the user
```

```
    number = float(input("Enter a Number: "))
```

```
    # Divide 100 by the entered number
```

```
    result = 100 / number
```

```
    print("Result =", result)
```

```
except (ValueError, ZeroDivisionError):
```

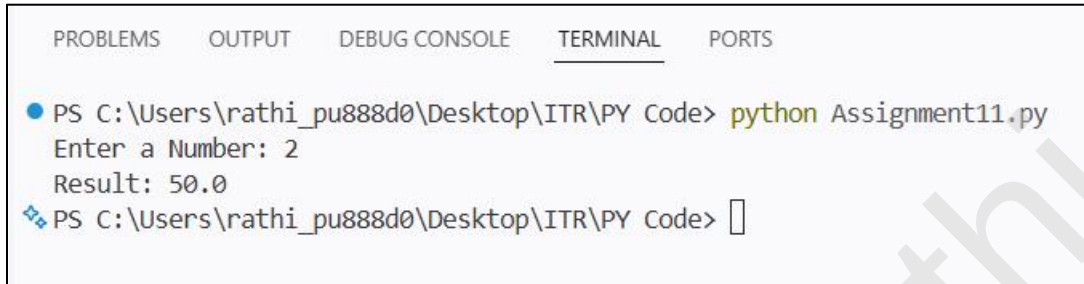
```
# Handle both exceptions together  
print("Error: Invalid input or division by zero.")
```

"Explanation in Comments"

```
# The program accepts a number from the user.  
# 100 is divided by the entered number.  
# ValueError occurs if the input is not numeric.  
# ZeroDivisionError occurs if the user enters 0.  
# Both exceptions are handled in a single except block  
# using a tuple: (ValueError, ZeroDivisionError).  
# A common error message is displayed for both cases.
```

Q9.Debugging Exception Code

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> python Assignment11.py
Enter a Number: 2
Result: 50.0
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> 
```

Program Code:

```
""" ASSIGNMENT 11 """
```

```
'''
```

Q9. (Debugging Exception Code)

The following code has issues.
Correct it and add comments explaining the fixes.

```
'''
```

```
try:
```

```
    # Take input from the user
    num = int(input("Enter a Number: "))
```

```
    # Divide 100 by the entered number
    result = 100 / num
```

```
    print("Result:", result)
```

```
except ValueError:
```

```
    # Handles non-numeric input
    print("Please enter a valid number.")
```

```
except ZeroDivisionError:
```

```
    # Handles division by zero
    print("Division by zero is not allowed.")
```

```
except Exception:
```

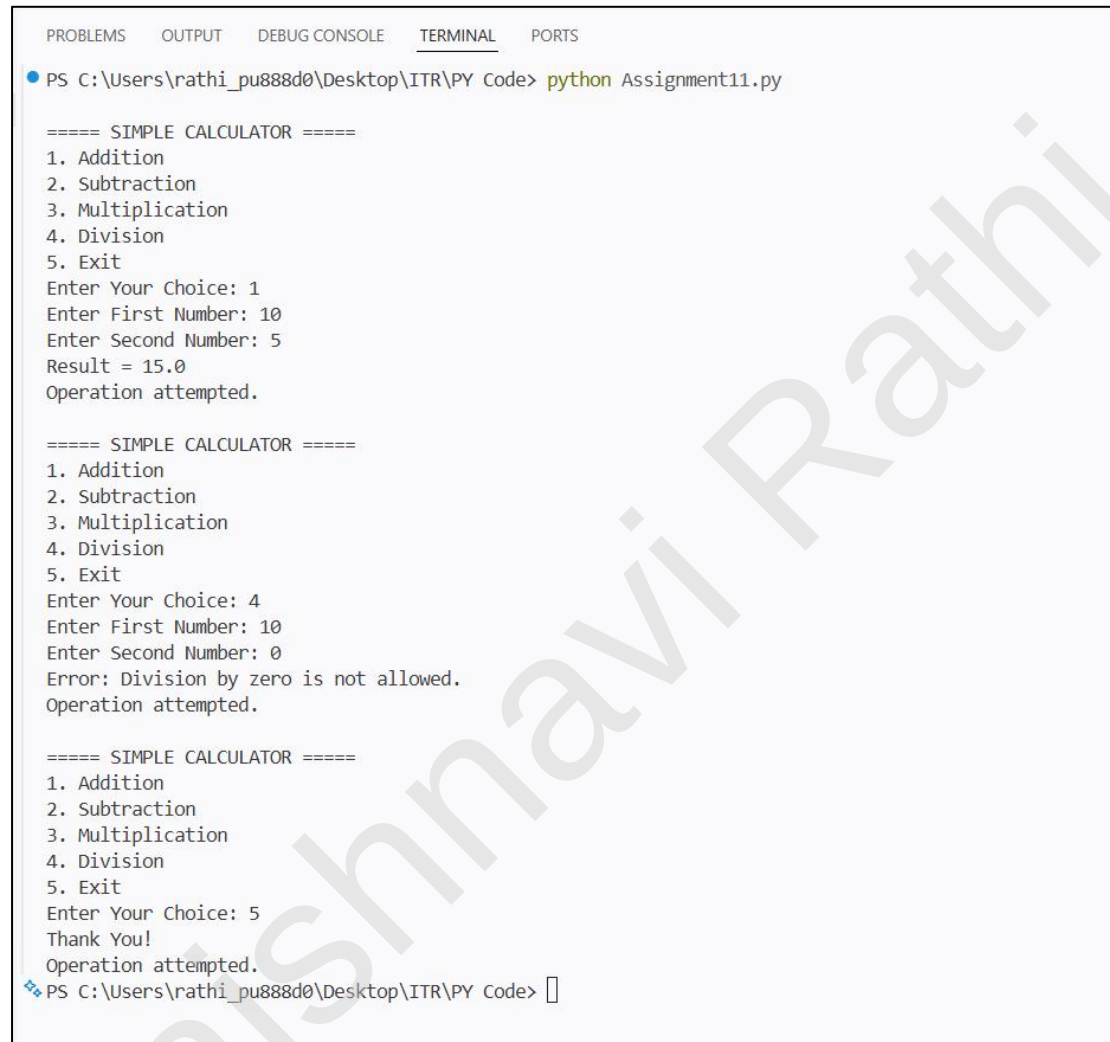
```
# Handles any other unexpected errors
print("Some error occurred.")
```

"Explanation in Comments"

```
# Fix 1: Added ':' after try and except statements.
# Fix 2: Proper indentation was added inside try and except blocks.
# Fix 3: Added ZeroDivisionError handling for division by zero.
# Fix 4: Used Exception instead of a bare except for better practice.
# Fix 5: Added comments to improve readability and understanding.
# The program now handles invalid input, division by zero,
# and other unexpected errors correctly.
```

Q10. Mini Project - Exception Handling

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> python Assignment11.py

===== SIMPLE CALCULATOR =====
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
Enter Your Choice: 1
Enter First Number: 10
Enter Second Number: 5
Result = 15.0
Operation attempted.

===== SIMPLE CALCULATOR =====
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
Enter Your Choice: 4
Enter First Number: 10
Enter Second Number: 0
Error: Division by zero is not allowed.
Operation attempted.

===== SIMPLE CALCULATOR =====
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
Enter Your Choice: 5
Thank You!
Operation attempted.
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> 
```

Program Code:

```
""" ASSIGNMENT 11 """
```

```
'''
```

Q10. (Mini Project - Exception Handling)

Create a Simple Calculator with exception handling.

Menu:

1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit

Handle:

- ValueError
- ZeroDivisionError

Use finally to print:

"Operation attempted."

```
'''
```

while True:

```
print("\n===== SIMPLE CALCULATOR =====")
print("1. Addition")
print("2. Subtraction")
print("3. Multiplication")
print("4. Division")
print("5. Exit")
```

```
try:
```

```
    choice = int(input("Enter Your Choice: "))
```

```
    if choice == 5:
```

```
        print("Thank You!")
```

```
        break
```

```
    num1 = float(input("Enter First Number: "))
```

```
    num2 = float(input("Enter Second Number: "))
```

```
    if choice == 1:
```

```

        print("Result =", num1 + num2)

    elif choice == 2:
        print("Result =", num1 - num2)

    elif choice == 3:
        print("Result =", num1 * num2)

    elif choice == 4:
        print("Result =", num1 / num2)

    else:
        print("Invalid Choice!")

except ValueError:

    # Handles invalid numeric input
    print("Error: Please enter valid numbers.")

except ZeroDivisionError:

    # Handles division by zero
    print("Error: Division by zero is not allowed.")

finally:

    # Executes after every operation
    print("Operation attempted.")

'''Explanation in Comments'''

# The program runs continuously using a while loop.
# A menu is displayed for calculator operations.
# User enters choice and two numbers.
# Calculations are performed inside the try block.
# ValueError handles invalid input.
# ZeroDivisionError handles division by zero.
# finally block executes after every operation.
# The loop ends only when the user selects Exit.

```
